ASSIGNMENT 2

Question 1– Hello Client-Server

SERVER CODE

```
#include <stdio.h>
#include <string.h>
#include <winsock2.h>
#define PORT 8080
int main()
{
   WSADATA wsa;
   SOCKET serverSocket, clientSocket;
   struct sockaddr_in serverAddr, clientAddr;
   int addrSize;
   char buffer[1024];
   WSAStartup(MAKEWORD(2, 2), &wsa);
   serverSocket = socket(AF_INET, SOCK_STREAM, 0);
   serverAddr.sin_family = AF_INET;
   serverAddr.sin_addr.s_addr = INADDR_ANY;
   serverAddr.sin_port = htons(PORT);
   bind(serverSocket, (struct sockaddr *)&serverAddr,
       sizeof(serverAddr));
   listen(serverSocket, 5);
   printf("Server is waiting for connection...\n");
   addrSize = sizeof(clientAddr);
```

```
    clientSocket = accept(serverSocket,
                          (struct sockaddr *)&clientAddr,
                          &addrSize);
    recv(clientSocket, buffer, sizeof(buffer), 0);

    printf("Client says: %s\n", buffer);
    strcpy(buffer, "Hello Client");
    send(clientSocket, buffer, strlen(buffer) + 1, 0);
    closesocket(clientSocket);
    closesocket(serverSocket);
    WSACleanup();
return 0;
}
```

CLIENT CODE

```
#include <stdio.h>
#include <string.h>
#include <winsock2.h>

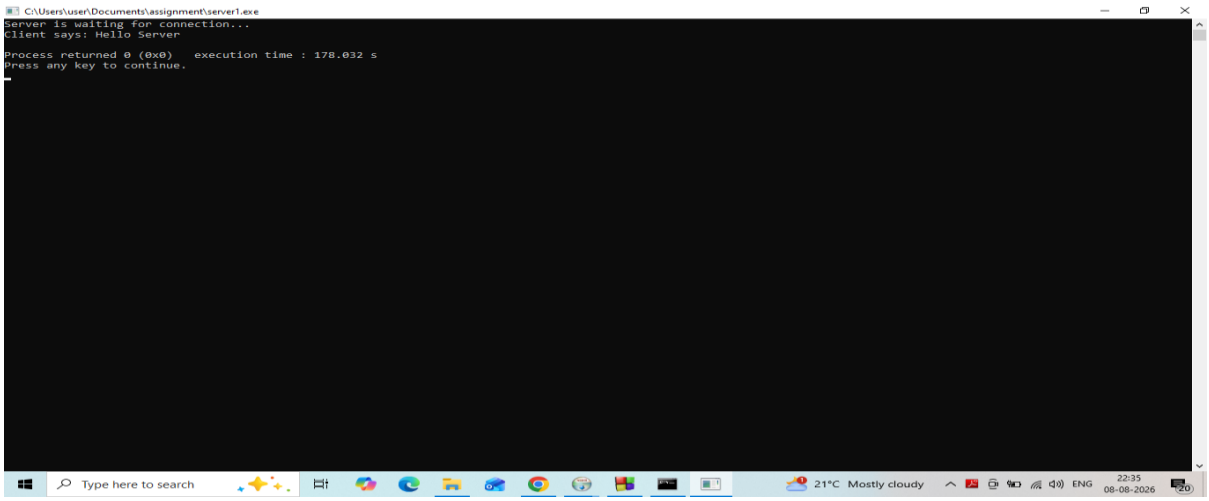
#define PORT 8080

int main()
{
    WSADATA wsa;
    SOCKET clientSocket;
    struct sockaddr_in serverAddr;
    char buffer[1024];
```

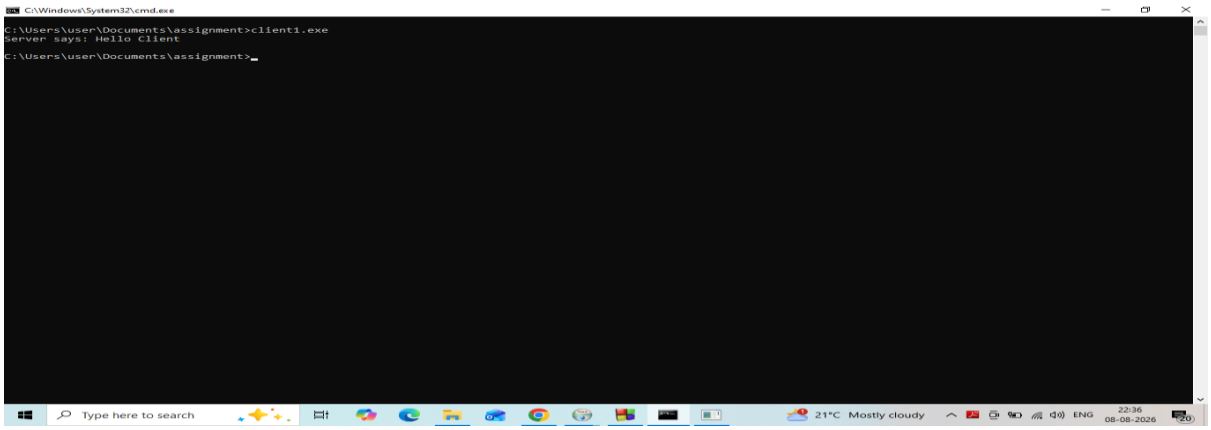
```
    WSAStartup(MAKEWORD(2, 2), &wsa);
    clientSocket = socket(AF_INET, SOCK_STREAM, 0);
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(PORT);
    serverAddr.sin_addr.s_addr = inet_addr("127.0.0.1");
    connect(clientSocket,
        (struct sockaddr *)&serverAddr,
        sizeof(serverAddr));
    strcpy(buffer, "Hello Server");
    send(clientSocket, buffer, strlen(buffer) + 1, 0);
    recv(clientSocket, buffer, sizeof(buffer), 0);

    printf("Server says: %s\n", buffer);
    closesocket(clientSocket);
    WSACleanup();
    return 0;
}
```

OUTPUT



Server Terminal



Client Terminal

Question 2– Arithmetic Server

SERVER CODE

```
#include <stdio.h>

#include <winsock2.h>


#define PORT 8080


int main()

{

    WSADATA wsa;

    SOCKET serverSocket, clientSocket;

    struct sockaddr_in serverAddr, clientAddr;

    int addrSize;


    int a, b;

    int sum, difference, product;

    float division;
```

```
// Initialize Winsock
WSAStartup(MAKEWORD(2, 2), &wsa);

// Create socket
serverSocket = socket(AF_INET, SOCK_STREAM, 0);

// Server address
serverAddr.sin_family = AF_INET;
serverAddr.sin_addr.s_addr = INADDR_ANY;
serverAddr.sin_port = htons(PORT);

// Bind
bind(serverSocket, (struct sockaddr *)&serverAddr,
    sizeof(serverAddr));

// Listen
listen(serverSocket, 5);

printf("Server is waiting for connection...\n");

// Accept client
addrSize = sizeof(clientAddr);

clientSocket = accept(serverSocket,
                (struct sockaddr *)&clientAddr,
                &addrSize);
```

```
printf("Client connected...\n");

// Receive two integers
recv(clientSocket, (char *)&a, sizeof(a), 0);
recv(clientSocket, (char *)&b, sizeof(b), 0);

// Perform calculations
sum = a + b;
difference = a - b;
product = a * b;

if (b != 0)
   division = (float)a / b;
else
   division = 0;

// Send results
send(clientSocket, (char *)&sum, sizeof(sum), 0);
send(clientSocket, (char *)&difference, sizeof(difference), 0);
send(clientSocket, (char *)&product, sizeof(product), 0);
send(clientSocket, (char *)&division, sizeof(division), 0);

closesocket(clientSocket);
closesocket(serverSocket);

WSACleanup();
```

```
    return 0;
}
```

CLIENT CODE

```
#include <stdio.h>
#include <winsock2.h>

#define PORT 8080

int main()
{
    WSADATA wsa;
    SOCKET clientSocket;
    struct sockaddr_in serverAddr;
    int a, b;
    int sum, difference, product;
    float division;
    WSAStartup(MAKEWORD(2, 2), &wsa);
    clientSocket = socket(AF_INET, SOCK_STREAM, 0);
    // Server address
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(PORT);
    serverAddr.sin_addr.s_addr = inet_addr("127.0.0.1");
    // Connect to server
    connect(clientSocket,
        (struct sockaddr *)&serverAddr,
```

```
        sizeof(serverAddr));
// Input
printf("Enter two integers: ");
scanf("%d %d", &a, &b);

// Send integers
send(clientSocket, (char *)&a, sizeof(a), 0);
send(clientSocket, (char *)&b, sizeof(b), 0);

// Receive results
recv(clientSocket, (char *)&sum, sizeof(sum), 0);
recv(clientSocket, (char *)&difference, sizeof(difference), 0);
recv(clientSocket, (char *)&product, sizeof(product), 0);
recv(clientSocket, (char *)&division, sizeof(division), 0);

// Display results
printf("\nAddition       = %d\n", sum);
printf("Subtraction    = %d\n", difference);
printf("Multiplication = %d\n", product);

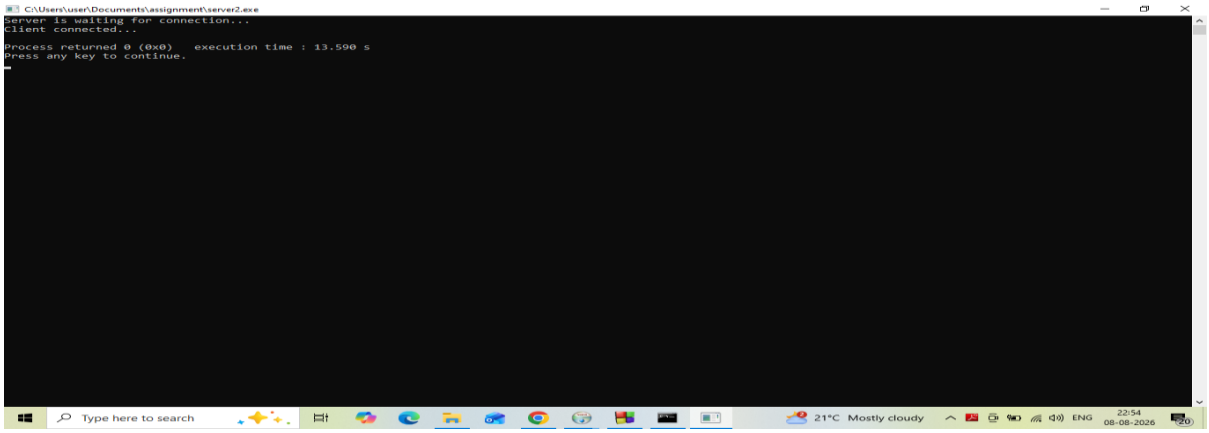
if (b != 0)
   printf("Division       = %.2f\n", division);
else
   printf("Division       = Cannot divide by zero\n");

closesocket(clientSocket);
```

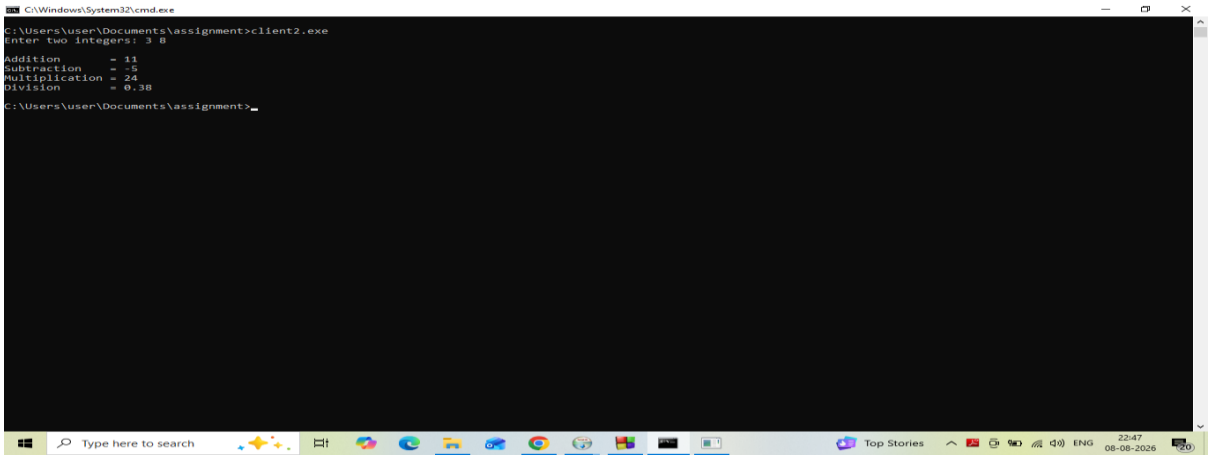
```
    WSACleanup();


    return 0;
}
```

OUTPUT



Server Terminal



Client Terminal

Question 3– String Processing Server

SERVER CODE

```
#include <stdio.h>

#include <string.h>

#include <ctype.h>
```

```
#include <winsock2.h>
#define PORT 8080
int main()
{
   WSADATA wsa;
   SOCKET serverSocket, clientSocket;
   struct sockaddr_in serverAddr, clientAddr;
   int addrSize;
   char str[1024];
   char uppercase[1024];
   char reverse[1024];
   int length;
   int i;
   WSAStartup(MAKEWORD(2, 2), &wsa);
   serverSocket = socket(AF_INET, SOCK_STREAM, 0);
   serverAddr.sin_family = AF_INET;
   serverAddr.sin_addr.s_addr = INADDR_ANY;
   serverAddr.sin_port = htons(PORT);
   bind(serverSocket, (struct sockaddr *)&serverAddr,
       sizeof(serverAddr));
   listen(serverSocket, 5);
   printf("Server is waiting for connection...\n");
   addrSize = sizeof(clientAddr);
   clientSocket = accept(serverSocket,
                         (struct sockaddr *)&clientAddr,
                         &addrSize);
```

```
    printf("Client connected...\n");
    recv(clientSocket, str, sizeof(str), 0);
    length = strlen(str);
    for (i = 0; i < length; i++)
    {
        uppercase[i] = toupper((unsigned char)str[i]);
    }
    uppercase[length] = '\0';
    for (i = 0; i < length; i++)
    {
        reverse[i] = str[length - 1 - i];
    }
    reverse[length] = '\0';
    send(clientSocket, (char *)&length, sizeof(length), 0);
    send(clientSocket, uppercase, sizeof(uppercase), 0);
    send(clientSocket, reverse, sizeof(reverse), 0);
    closesocket(clientSocket);
    closesocket(serverSocket);
    WSACleanup();
    return 0;
}
```

CLIENT CODE

```
#include <stdio.h>
#include <string.h>
#include <winsock2.h>
#define PORT 8080
```

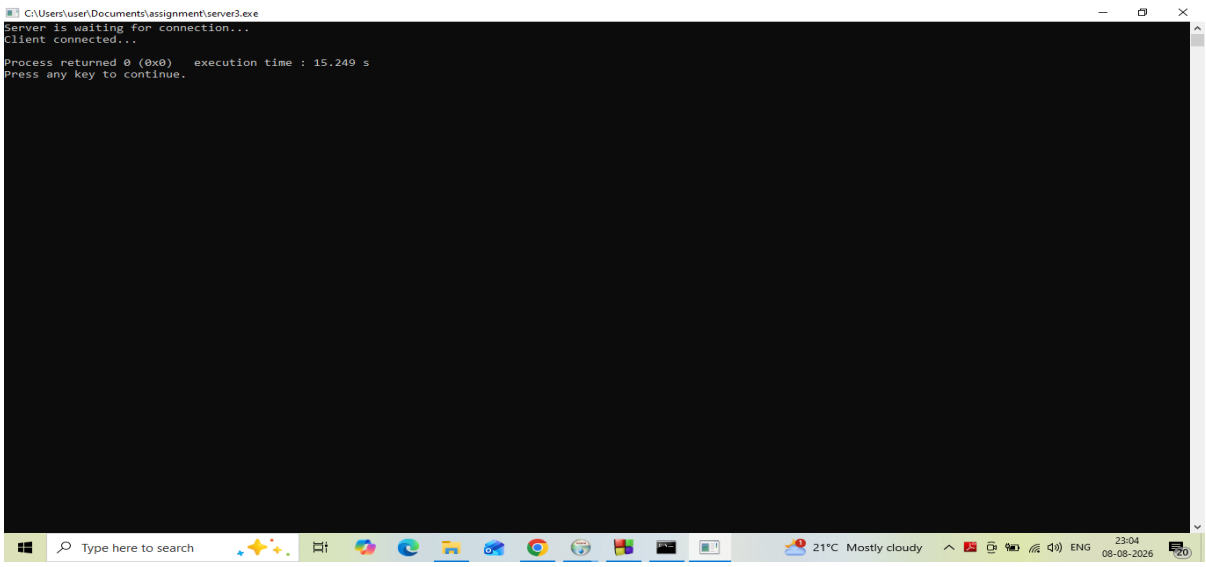
```
int main()
{
    WSADATA wsa;
    SOCKET clientSocket;
    struct sockaddr_in serverAddr;
    char str[1024];
    char uppercase[1024];
    char reverse[1024];
    int length;

    // Initialize Winsock
    WSAStartup(MAKEWORD(2, 2), &wsa);
    // Create socket
    clientSocket = socket(AF_INET, SOCK_STREAM, 0);
    // Server address
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(PORT);
    serverAddr.sin_addr.s_addr = inet_addr("127.0.0.1");
    // Connect to server
    connect(clientSocket,
        (struct sockaddr *)&serverAddr,
        sizeof(serverAddr));
    // Input string
    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);
    // Remove newline
```

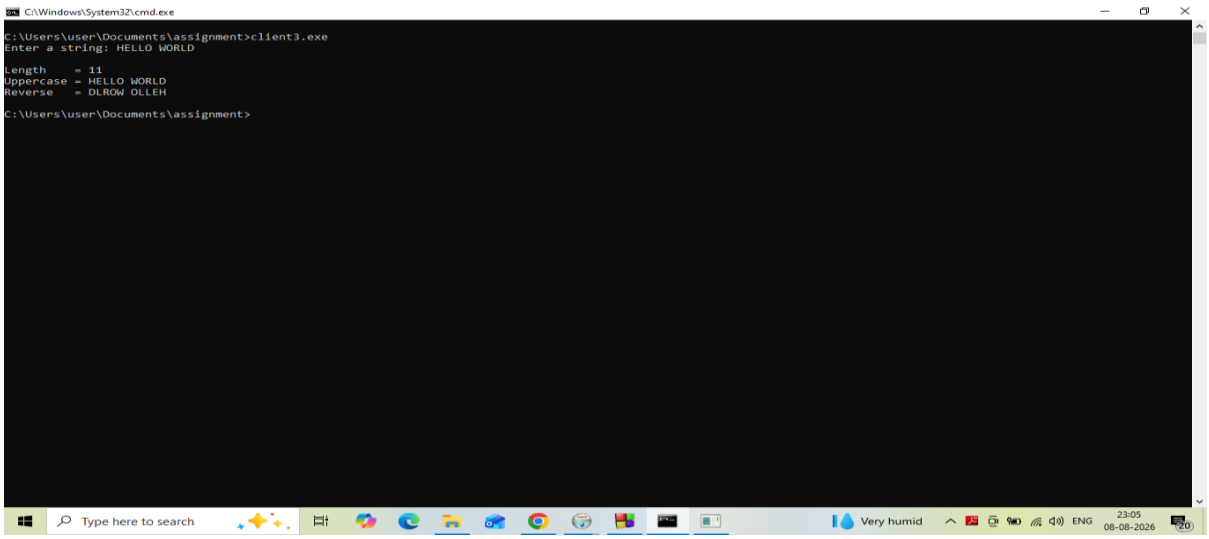
```
    str[strcspn(str, "\n")] = '\0';
    // Send string
    send(clientSocket, str, sizeof(str), 0);
    // Receive results
    recv(clientSocket, (char *)&length, sizeof(length), 0);
    recv(clientSocket, uppercase, sizeof(uppercase), 0);
    recv(clientSocket, reverse, sizeof(reverse), 0);
    // Display results
    printf("\nLength    = %d\n", length);
    printf("Uppercase = %s\n", uppercase);
    printf("Reverse   = %s\n", reverse);

    closesocket(clientSocket);
    WSACleanup();
    return 0;
}
```

OUTPUT



Server Terminal



Client Terminal

Question 4– Array Processing Server

SERVER CODE

```
#include <stdio.h>

#include <stdlib.h>

#include <winsock2.h>


#pragma comment(lib, "ws2_32.lib")


#define PORT 8080


int recvAll(SOCKET sock, char *buffer, int size)

{

    int total = 0;

    int bytes;


    while (total < size)

    {
```

```
        bytes = recv(sock, buffer + total, size - total, 0);

        if (bytes <= 0)
            return 0;

        total += bytes;
    }

    return 1;
}

int main()
{
    WSADATA wsa;
    SOCKET serverSocket, clientSocket;
    struct sockaddr_in server, client;
    int clientSize;
    int n, i;
    int arr[100];

    int sum, maximum, minimum;
    double average;

    /* Start Winsock */
    WSAStartup(MAKEWORD(2, 2), &wsa);
```

```
/* Create socket */
serverSocket = socket(AF_INET, SOCK_STREAM, 0);

if (serverSocket == INVALID_SOCKET)
{
   printf("Socket creation failed.\n");
   return 1;
}

/* Server address */
server.sin_family = AF_INET;
server.sin_addr.s_addr = INADDR_ANY;
server.sin_port = htons(PORT);

/* Bind */
if (bind(serverSocket, (struct sockaddr *)&server,
        sizeof(server)) == SOCKET_ERROR)
{
   printf("Bind failed.\n");
   closesocket(serverSocket);
   WSACleanup();
   return 1;
}

/* Listen */
listen(serverSocket, 1);
```

```
printf("Server waiting for client...\n");

/* Accept client */
clientSize = sizeof(client);
clientSocket = accept(serverSocket,
                      (struct sockaddr *)&client,
                      &clientSize);

if (clientSocket == INVALID_SOCKET)
{
   printf("Accept failed.\n");
   closesocket(serverSocket);
   WSACleanup();
   return 1;
}

printf("Client connected.\n");

/* Receive N */
if (!recvAll(clientSocket, (char *)&n, sizeof(int)))
{
   printf("Error receiving N.\n");
   closesocket(clientSocket);
   closesocket(serverSocket);
   WSACleanup();
```

```
    return 1;
}

/* Receive array */
if (!recvAll(clientSocket, (char *)arr, n * sizeof(int)))
{
    printf("Error receiving array.\n");
    closesocket(clientSocket);
    closesocket(serverSocket);
    WSACleanup();
    return 1;
}

/* Calculate results */
sum = 0;
maximum = arr[0];
minimum = arr[0];

for (i = 0; i < n; i++)
{
    sum += arr[i];

    if (arr[i] > maximum)
        maximum = arr[i];

    if (arr[i] < minimum)
```

```
            minimum = arr[i];
        }

        average = (double)sum / n;

        /* Send results */
        send(clientSocket, (char *)&sum, sizeof(int), 0);
        send(clientSocket, (char *)&average, sizeof(double), 0);
        send(clientSocket, (char *)&maximum, sizeof(int), 0);
        send(clientSocket, (char *)&minimum, sizeof(int), 0);

        printf("Results sent to client.\n");

        /* Close sockets */
        closesocket(clientSocket);
        closesocket(serverSocket);
        WSACleanup();

        return 0;
}
```

CLIENT CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <winsock2.h>

#pragma comment(lib, "ws2_32.lib")
```

```
#define PORT 8080

int recvAll(SOCKET sock, char *buffer, int size)
{
    int total = 0;
    int bytes;

    while (total < size)
    {
        bytes = recv(sock, buffer + total, size - total, 0);

        if (bytes <= 0)
            return 0;

        total += bytes;
    }

    return 1;
}

int main()
{
    WSADATA wsa;
    SOCKET clientSocket;
    struct sockaddr_in server;
```

```
int n, i;
int arr[100];

int sum, maximum, minimum;
double average;

/* Start Winsock */
WSAStartup(MAKEWORD(2, 2), &wsa);

/* Create socket */
clientSocket = socket(AF_INET, SOCK_STREAM, 0);

if (clientSocket == INVALID_SOCKET)
{
   printf("Socket creation failed.\n");
   return 1;
}

/* Server address */
server.sin_family = AF_INET;
server.sin_addr.s_addr = inet_addr("127.0.0.1");
server.sin_port = htons(PORT);

/* Connect to server */
if (connect(clientSocket,
```

```
        (struct sockaddr *)&server,
        sizeof(server)) == SOCKET_ERROR)
{
   printf("Connection failed.\n");
   closesocket(clientSocket);
   WSACleanup();
   return 1;
}

printf("Connected to server.\n");

/* Input N */
printf("Enter number of integers: ");
scanf("%d", &n);

if (n <= 0 || n > 100)
{
   printf("Enter N between 1 and 100.\n");
   closesocket(clientSocket);
   WSACleanup();
   return 1;
}

/* Input array */
printf("Enter %d integers:\n", n);
```

```
for (i = 0; i < n; i++)
{
   scanf("%d", &arr[i]);
}

/* Send N */
send(clientSocket, (char *)&n, sizeof(int), 0);

/* Send array */
send(clientSocket, (char *)arr, n * sizeof(int), 0);

/* Receive results */
if (!recvAll(clientSocket, (char *)&sum, sizeof(int)) ||
   !recvAll(clientSocket, (char *)&average, sizeof(double)) ||
   !recvAll(clientSocket, (char *)&maximum, sizeof(int)) ||
   !recvAll(clientSocket, (char *)&minimum, sizeof(int)))
{
   printf("Error receiving results.\n");
   closesocket(clientSocket);
   WSACleanup();
   return 1;
}

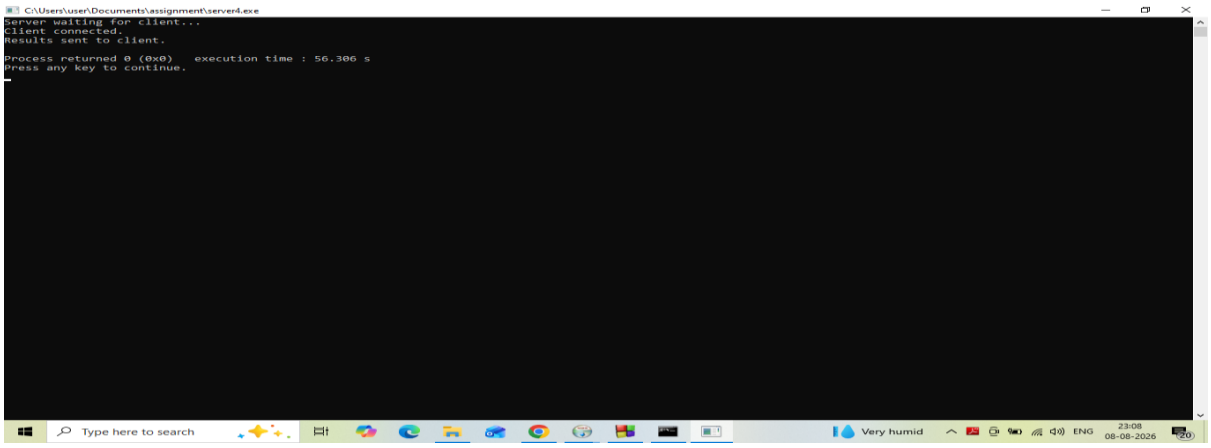
/* Display results */
printf("\n----- Results -----\n");
printf("Sum     = %d\n", sum);
```

```
    printf("Average = %.2f\n", average);
    printf("Maximum = %d\n", maximum);
    printf("Minimum = %d\n", minimum);

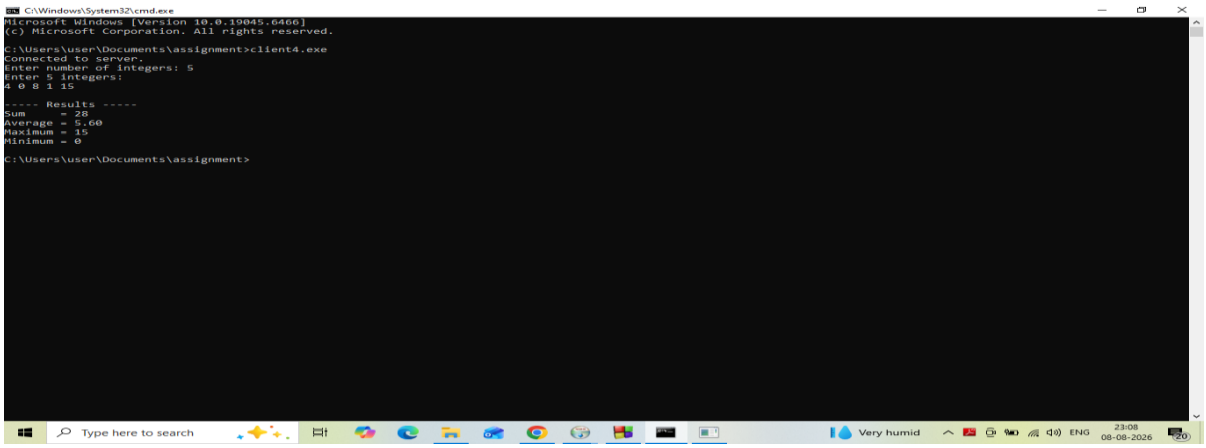
    /* Close socket */
    closesocket(clientSocket);
    WSACleanup();

    return 0;
}
```

OUTPUT



Server Terminal



Client Terminal

Question 5– Matrix Analyzer

SERVER CODE

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <winsock2.h>

#include <ws2tcpip.h>


#define PORT 8080

#define MAX 10


/* Function to receive exactly n bytes */

int receiveAll(SOCKET socket, char *buffer, int size)

{

   int total = 0;

   int bytes;


   while (total < size)

   {

      bytes = recv(socket, buffer + total, size - total, 0);


      if (bytes <= 0)

         return 0;


      total += bytes;

   }
```

```
    return 1;
}

int main()
{
    WSADATA wsa;
    SOCKET serverSocket, clientSocket;

    struct sockaddr_in serverAddr;

    int n;
    int matrix[MAX][MAX];

    int i, j;
    int sum = 0;
    int trace = 0;

    int isDiagonal = 1;
    int isUpper = 1;
    int isLower = 1;

    char matrixType[50];

    /* Initialize Winsock */
```

```
if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0)
{
   printf("WSAStartup Failed\n");
   return 1;
}

printf("Winsock Initialized Successfully...\n");

/* Create Socket */

serverSocket = socket(AF_INET, SOCK_STREAM, 0);

if (serverSocket == INVALID_SOCKET)
{
   printf("Socket Creation Failed\n");
   WSACleanup();
   return 1;
}

printf("Socket Created Successfully...\n");

/* Server Address */

serverAddr.sin_family = AF_INET;
serverAddr.sin_addr.s_addr = INADDR_ANY;
serverAddr.sin_port = htons(PORT);
```

```
/* Bind */

if (bind(serverSocket,
        (struct sockaddr *)&serverAddr,
        sizeof(serverAddr)) == SOCKET_ERROR)
{
   printf("Bind Failed\n");
   closesocket(serverSocket);
   WSACleanup();
   return 1;
}

printf("Bind Successful...\n");

/* Listen */

if (listen(serverSocket, 5) == SOCKET_ERROR)
{
   printf("Listen Failed\n");
   closesocket(serverSocket);
   WSACleanup();
   return 1;
}

printf("Waiting for Client...\n");
```

```
/* Accept */

clientSocket = accept(serverSocket, NULL, NULL);

if (clientSocket == INVALID_SOCKET)
{
   printf("Accept Failed\n");
   closesocket(serverSocket);
   WSACleanup();
   return 1;
}

printf("Client Connected...\n");

/* Receive matrix order */

if (!receiveAll(clientSocket, (char *)&n, sizeof(n)))
{
   printf("Failed to receive matrix order\n");
   closesocket(clientSocket);
   closesocket(serverSocket);
   WSACleanup();
   return 1;
}
```

```
printf("Matrix Order Received: %d\n", n);

/* Receive matrix */

if (!receiveAll(clientSocket,
            (char *)matrix,
            sizeof(int) * n * n))
{
   printf("Failed to receive matrix\n");
   closesocket(clientSocket);
   closesocket(serverSocket);
   WSACleanup();
   return 1;
}

/* Display Matrix */

printf("\nMatrix Received:\n");

for (i = 0; i < n; i++)
{
   for (j = 0; j < n; j++)
   {
      printf("%d\t", matrix[i][j]);
   }
   printf("\n");
```

```
}

/* Matrix Analysis */

for (i = 0; i < n; i++)
{
   for (j = 0; j < n; j++)
   {
      /* Calculate Sum */

      sum = sum + matrix[i][j];

      /* Calculate Trace */

      if (i == j)
      {
         trace = trace + matrix[i][j];
      }

      /* Check Diagonal */

      if (i != j && matrix[i][j] != 0)
      {
         isDiagonal = 0;
      }
```

```
        /* Check Upper Triangular */

        if (i > j && matrix[i][j] != 0)
        {
            isUpper = 0;
        }

        /* Check Lower Triangular */

        if (i < j && matrix[i][j] != 0)
        {
            isLower = 0;
        }
    }
}

/* Determine Matrix Type */

if (isDiagonal)
{
    strcpy(matrixType, "Diagonal Matrix");
}
else if (isUpper)
{
    strcpy(matrixType, "Upper Triangular Matrix");
}
```

```
else if (isLower)
{
   strcpy(matrixType, "Lower Triangular Matrix");
}
else
{
   strcpy(matrixType, "General Matrix");
}

printf("\nMatrix Type : %s\n", matrixType);
printf("Sum        : %d\n", sum);
printf("Trace      : %d\n", trace);

/* Send Matrix Type */

send(clientSocket,
    matrixType,
    sizeof(matrixType),
    0);

/* Send Sum */

send(clientSocket,
    (char *)&sum,
    sizeof(sum),
    0);
```

```
    /* Send Trace */

    send(clientSocket,
        (char *)&trace,
        sizeof(trace),
        0);

    printf("\nResults Sent to Client...\n");

    /* Close */

    closesocket(clientSocket);
    closesocket(serverSocket);

    WSACleanup();

    printf("Connection Closed...\n");

    return 0;
}
```

CLIENT CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <winsock2.h>
```

```
#include <ws2tcpip.h>

#define PORT 8080
#define MAX 10

/* Function to receive exactly n bytes */
int receiveAll(SOCKET socket, char *buffer, int size)
{
   int total = 0;
   int bytes;

   while (total < size)
   {
      bytes = recv(socket, buffer + total, size - total, 0);

      if (bytes <= 0)
         return 0;

      total += bytes;
   }

   return 1;
}

int main()
{
```

```
WSADATA wsa;
SOCKET sockfd;

struct sockaddr_in serverAddr;

int n;
int matrix[MAX][MAX];

int i, j;

char matrixType[50];
int sum;
int trace;

/* Initialize Winsock */

if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0)
{
   printf("WSAStartup Failed\n");
   return 1;
}

printf("Winsock Initialized Successfully...\n");

/* Create Socket */
```

```
sockfd = socket(AF_INET, SOCK_STREAM, 0);

if (sockfd == INVALID_SOCKET)
{
   printf("Socket Creation Failed\n");
   WSACleanup();
   return 1;
}

printf("Socket Created Successfully...\n");

/* Server Address */

serverAddr.sin_family = AF_INET;
serverAddr.sin_port = htons(PORT);

inet_pton(AF_INET,
        "127.0.0.1",
        &serverAddr.sin_addr);

/* Connect */

if (connect(sockfd,
          (struct sockaddr *)&serverAddr,
          sizeof(serverAddr)) == SOCKET_ERROR)
{
```

```
    printf("Connection Failed\n");

    closesocket(sockfd);
    WSACleanup();

    return 1;
}

printf("Connected to Server...\n");

/* Input Matrix Order */

printf("\nEnter the order of matrix N (maximum %d): ", MAX);
scanf("%d", &n);

if (n <= 0 || n > MAX)
{
    printf("Invalid matrix order!\n");

    closesocket(sockfd);
    WSACleanup();

    return 1;
}

/* Input Matrix */
```

```
printf("\nEnter the elements of the matrix:\n");

for (i = 0; i < n; i++)
{
   for (j = 0; j < n; j++)
   {
      scanf("%d", &matrix[i][j]);
   }
}

/* Display Matrix */

printf("\nMatrix Entered:\n");

for (i = 0; i < n; i++)
{
   for (j = 0; j < n; j++)
   {
      printf("%d\t", matrix[i][j]);
   }

   printf("\n");
}

/* Send Matrix Order */
```

```
send(sockfd,
    (char *)&n,
    sizeof(n),
    0);

/* Send Matrix */

send(sockfd,
    (char *)matrix,
    sizeof(int) * n * n,
    0);

printf("\nMatrix Sent to Server...\n");

/* Receive Matrix Type */

if (!receiveAll(sockfd,
            matrixType,
            sizeof(matrixType)))
{
   printf("Failed to receive matrix type\n");

   closesocket(sockfd);
   WSACleanup();
```

```
    return 1;
}

/* Receive Sum */

if (!receiveAll(sockfd,
                (char *)&sum,
                sizeof(sum)))
{
    printf("Failed to receive sum\n");

    closesocket(sockfd);
    WSACleanup();

    return 1;
}

/* Receive Trace */

if (!receiveAll(sockfd,
                (char *)&trace,
                sizeof(trace)))
{
    printf("Failed to receive trace\n");

    closesocket(sockfd);
```

```
        WSACleanup();

        return 1;
    }

    /* Display Results */

    printf("\n================================\n");
    printf("       MATRIX ANALYSIS RESULT\n");
    printf("================================\n");

    printf("Matrix Type : %s\n", matrixType);
    printf("Sum         : %d\n", sum);
    printf("Trace       : %d\n", trace);

    printf("================================\n");

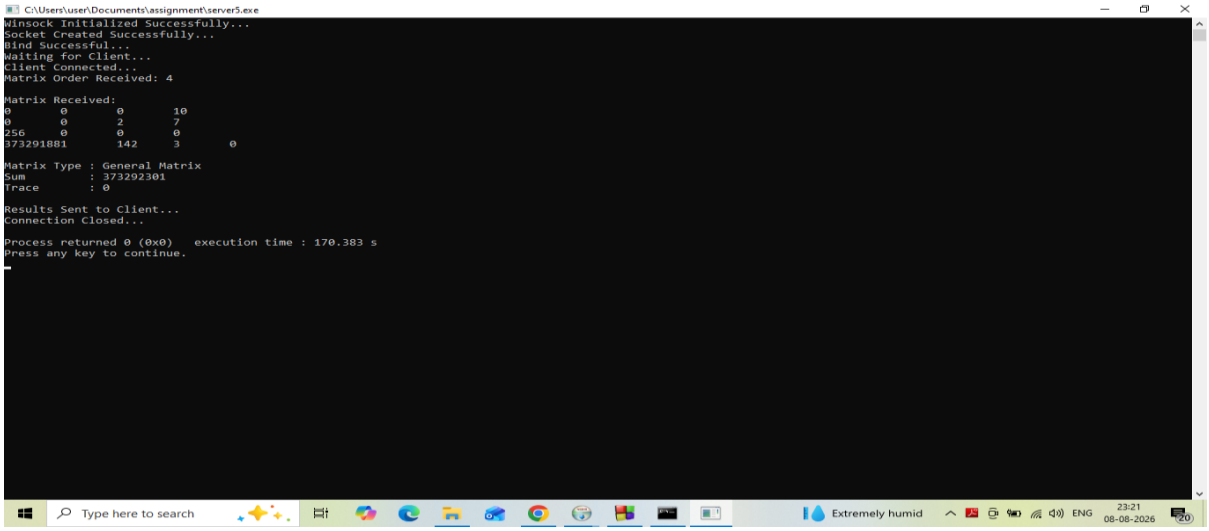
    /* Close */

    closesocket(sockfd);

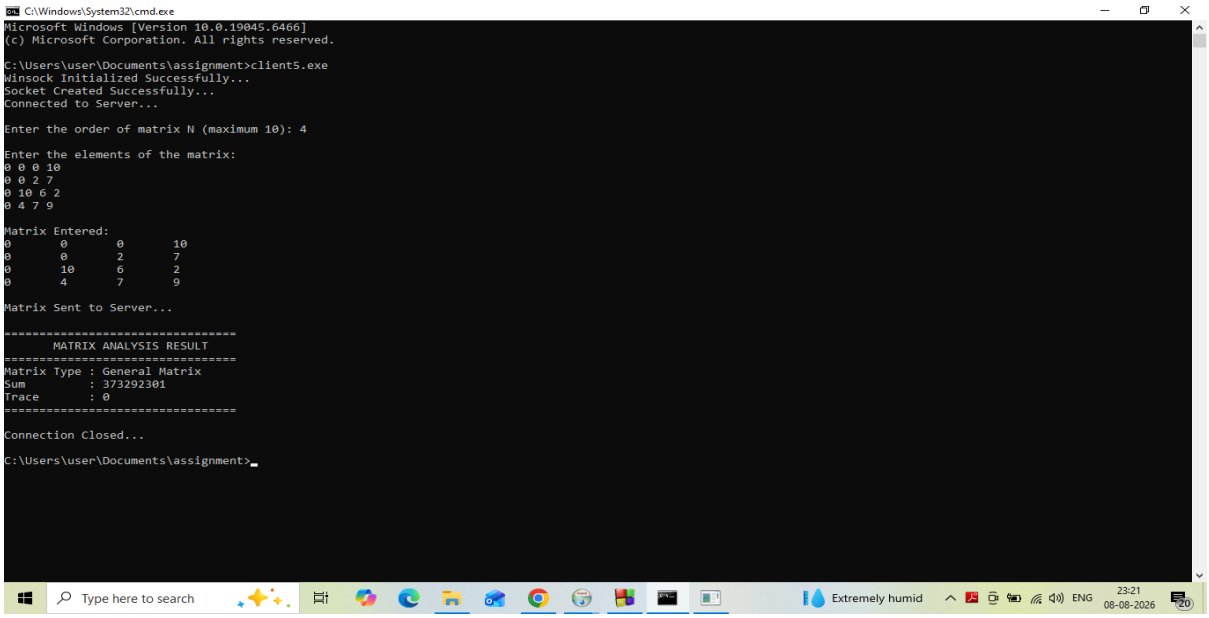
    WSACleanup();

    printf("\nConnection Closed...\n");
    return 0;
}
```

OUTPUT



Server Terminal



Client Terminal